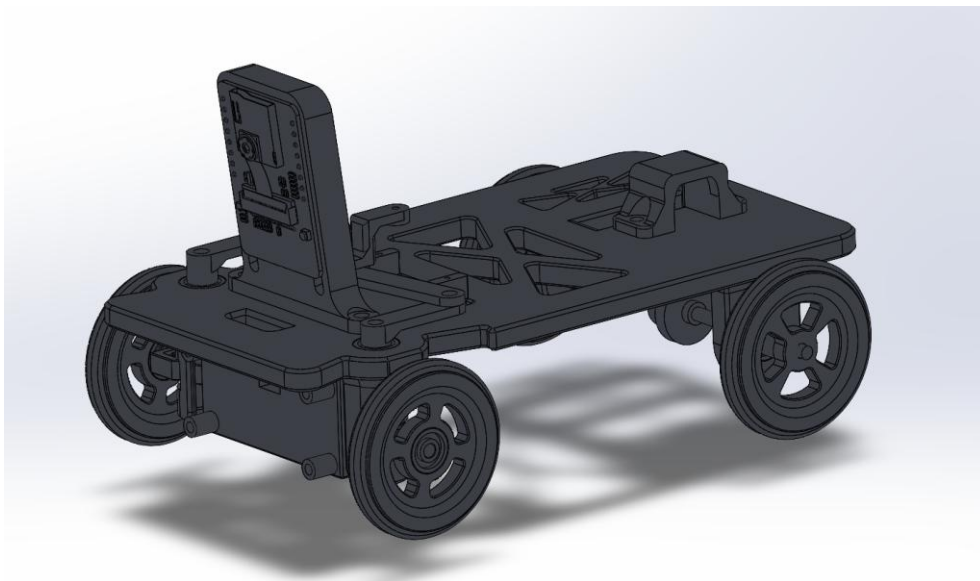


WRO 2026

Future Engineers

Team CYBERRCORE



Full Robot Assembly — Isometric View

TECHNICAL ENGINEERING DOCUMENTATION

Document version: June 2026 | Confidential — WRO Competition Use Only

Table of Contents

1 System Overview	3
2 Mechanical Design	4
2.1 Main Chassis	4
2.2 Steering System	5
2.3 Rear Drivetrain & Differential	6
2.4 Wheel & Tire System	7
2.5 Sensor & Electronics Mounts	8
3 Electronics & Wiring	9
3.1 Bill of Materials	9
3.2 Cable & Connection Specifications	10
3.3 Power Budget	12
4 Software Architecture	13
4.1 Vision System (ESP32-CAM)	13
4.2 Open Round Navigation	14
4.3 Obstacle Round Navigation	15
4.4 Control System (PID)	16
5 Vehicle Photos	17
6 Validation & Testing	19
7 Engineering Decisions	20

1 System Overview

Team CYBERRCORE's WRO 2026 Future Engineers robot is a fully autonomous wheeled vehicle designed for the open-round and obstacle-round competition formats. The system integrates a Raspberry Pi Pico 2 W H main controller with an ESP32-CAM vision module, a TB6612FNG brushed DC motor driver, a HD-1440A steering servo, US-100 ultrasonic distance sensor, and MPU6050 IMU — all powered from a 2S 450 mAh LiPo battery through dual LM2596 buck converter rails.

All mechanical components were custom-designed in SolidWorks and 3D-printed. The chassis uses a flat-plate architecture with X-pattern lightening cutouts, Ackermann front steering linkage, and a 1:1 spur-gear differential rear drivetrain. The software stack is split between Arduino C++ firmware on the ESP32-CAM for color detection and MicroPython scripts on the Pico for navigation, motor control, and sensor fusion.

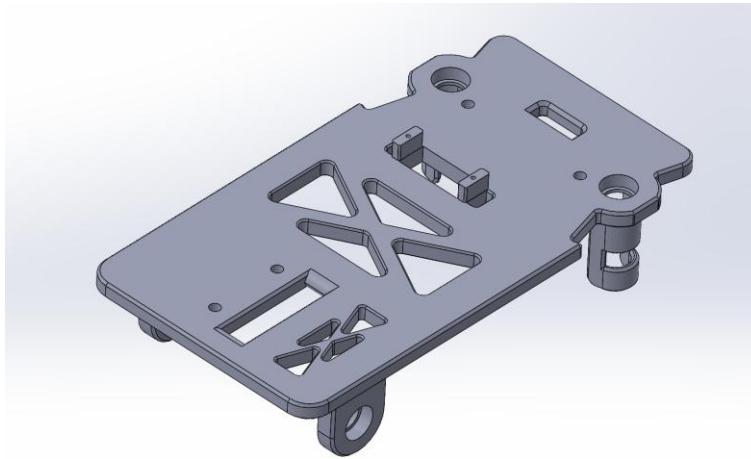
Subsystem	Technology	Role
Main Controller	Raspberry Pi Pico 2 W H	MicroPython navigation, PID steering, sensor fusion
Vision Module	ESP32-CAM (Arduino C++)	RGB565 color detection, UART obstacle reporting
Motor Driver	TB6612FNG	PWM drive control for brushed DC motor
Drive Motor	6 V Micro DC Motor	Rear wheel propulsion via 1:1 spur gear
Steering	HD-1440A Servo	Ackermann front wheel steering
Distance Sensing	US-100 Ultrasonic	Wall detection, corner trigger, safety stop
IMU / Gyro	MPU6050	Yaw estimation, 90° turn completion, heading recovery
Color Sensor	TCS34725	Backup / future color validation
Power Source	2S 450 mAh 40C LiPo	Main battery — 7.4 V nominal
Regulation	2× LM2596 Buck Converter	5 V logic rail + 5/6 V servo rail
Chassis / Structure	SolidWorks + FDM 3D Print	All mechanical parts custom-designed and printed

2 Mechanical Design

The mechanical platform is designed around structural rigidity with strategic weight reduction. A flat-plate chassis architecture keeps the center of gravity low, simplifies assembly, and provides dedicated mounting zones for every subsystem. All parts were modeled parametrically in SolidWorks and 3D-printed in PLA for prototyping with PETG/ABS targeted for competition.

2.1 Main Chassis

The primary structural plate features X-pattern lightening pockets that remove mass where structurally unnecessary while preserving load paths. Integrated standoff columns allow direct PCB mounting without separate hardware. Distinct zones are pre-allocated for motor mount, servo, differential, and sensor brackets.

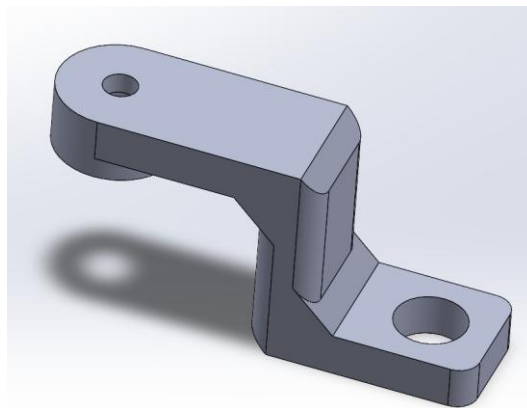


Main chassis — X-pattern lightening pockets and integrated standoff columns

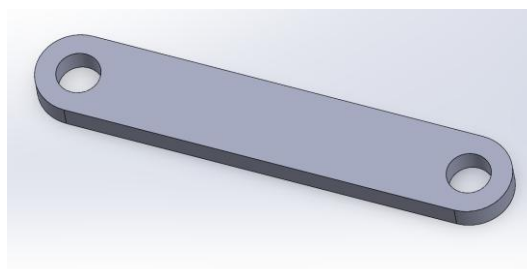
Feature	Detail
Architecture	Single-level flat plate — low CG for stability at speed
Lightening Cutouts	X-pattern triangular pocket geometry
Standoff Columns	Molded-in posts for direct PCB and cover mounting
Subsystem Zones	Dedicated areas for motor, servo, differential, sensor brackets
Perimeter Rail	Continuous edge flange for external bracket attachment
Print Settings	40–60% infill, gyroid pattern, layer lines $\perp$ to primary stress

2.2 Steering System — 4-Part Ackermann Linkage

The front steering uses a four-component Ackermann-compliant linkage ensuring both front wheels track toward a common turn center during cornering. The asymmetric offset geometry of the steering arms ensures the inner wheel turns through a greater angle than the outer wheel, eliminating tire scrub.



Steer1 — Steering Knuckle: L-shaped bracket connecting wheel spindle to chassis pivot

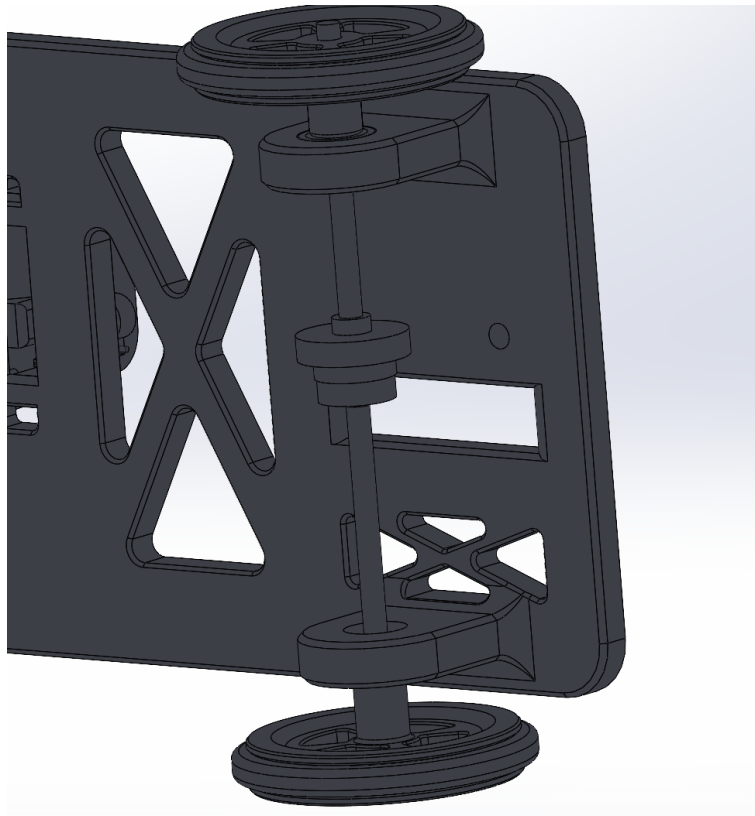


Steer2 — Tie Rod: flat bar transmitting servo motion to front knuckles

Part	Role	Key Feature
Steer1 — Knuckle	Connects wheel spindle to chassis upright	L-shape; upper hole for servo pin, lower for chassis pivot
Steer2 — Tie Rod	Transmits servo motion to wheel knuckles	Equal-diameter holes at each end, backlash-free length
Steer3 — Left Arm	Translates servo rotation to wheel angle	Asymmetric boss; compensates Ackermann geometry (left)
Steer4 — Right Arm	Mirror function for right wheel	Cylindrical boss for shaft engagement; mirrored to Steer3

2.3 Rear Drivetrain & Differential

The rear drivetrain uses a 1:1 spur-gear pair (31T–31T, 0.8 module) to transfer motor torque to a through-axle supported by 3×10×4 bearings. A differential allows independent rear wheel rotation during turns, preventing inside-wheel scrub and improving corner repeatability.



Assembled differential — rear shaft, gear stack, and bearing assembly

Parameter	Value	Justification
Gear module	0.8M	Compact pitch suited for small-scale precision drivetrains
Tooth count	31T : 31T	1:1 ratio — preserves full motor RPM
Drive ratio	1:1	Prioritises speed; torque sufficient for vehicle mass
Bearing type	3×10×4 radial ball	Reduces friction; consistent over repeated laps
Differential	Custom printed	Allows independent rear wheel speeds in corners

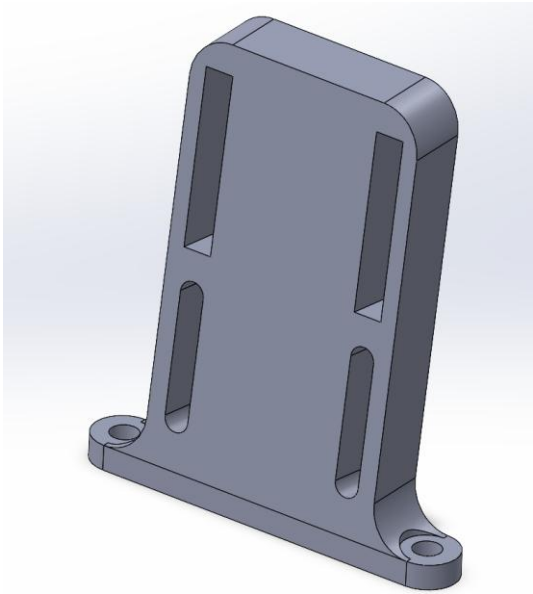
2.4 Wheel & Tire System

Front and rear wheels have distinct designs to match their functional roles. O-ring style tires press onto the outer rim flange, providing simple replacement and consistent rolling contact patch.

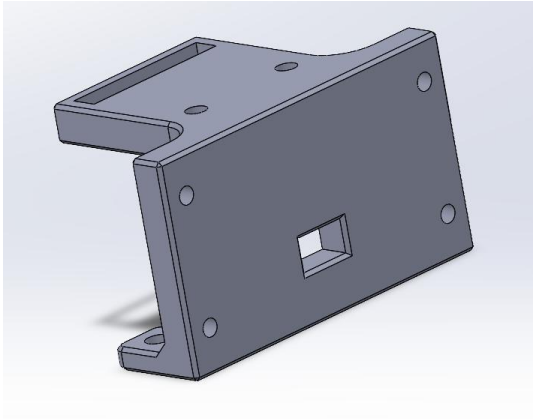
Aspect	Front Wheel	Rear Wheel
Hub Type	Recessed bearing seat (press-fit 3×10×4)	Direct shaft hub — no bearing
Spoke Pattern	4 spokes with intermediate web	4 solid spokes, lighter
Function	Free rotation around fixed spindle	Fixed to axle shaft, driven rotationally
Tire Fit	O-ring press-fit over rim flange	O-ring press-fit over rim flange

Tire profile: O-ring torus with three parallel circumferential grip grooves. Material: flexible rubber/TPU. Advantages: tool-free replacement, consistent geometry batch-to-batch.

2.5 Sensor & Electronics Mounts



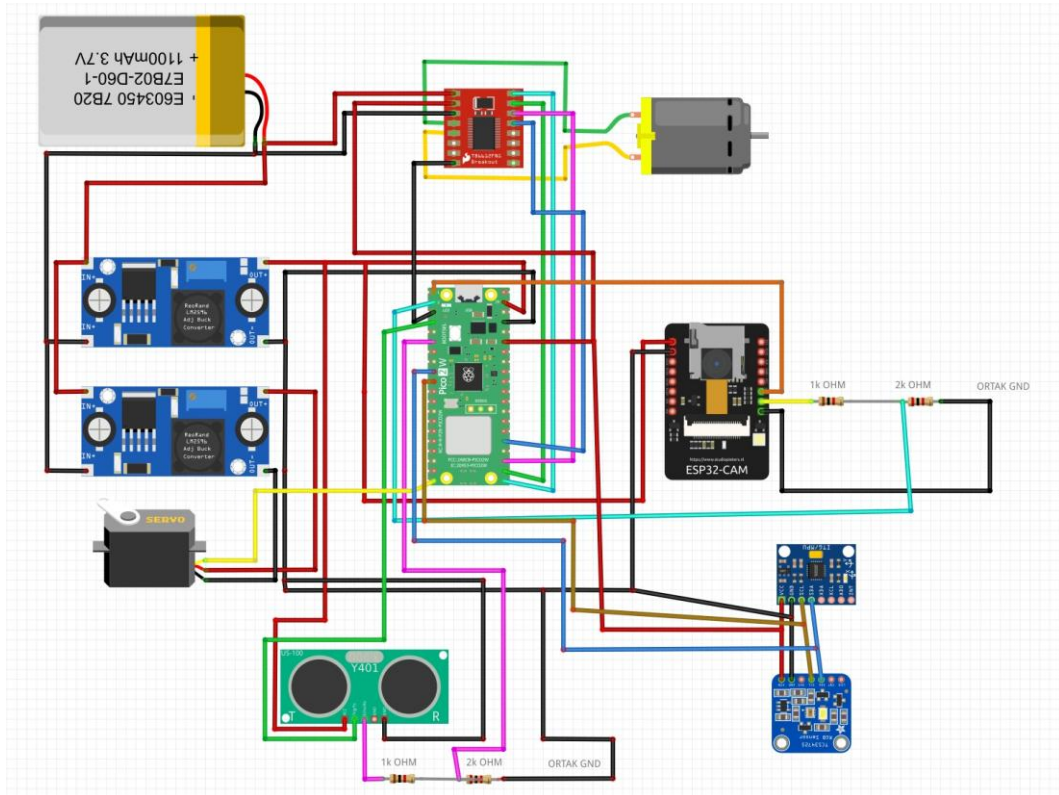
Camera Mount — tall vertical bracket with dual height-adjustment slots



US-100 Mount — box bracket with square sensor cutout and snap-clip retention

Mount	Key Features
Camera Mount (CamMount)	Tall vertical bracket; 2 vertical adjustment slots for height fine-tuning; angled forward lean for near-field obstacle detection; 2 M3 base holes
US-100 Mount (US100_Mount)	Box-form bracket; square cutout sized to US-100 body; side snap-tab — no fasteners; 4 corner mounting holes; compact to minimise aerodynamic interference
Motor Mount (motorMount)	U-bracket wrapping motor on 3 sides; snap-clip for tool-free removal; 2 M3 base holes; gear alignment built into mount geometry

3 Electronics & Wiring



Complete electronics wiring scheme

3.1 Bill of Materials

Component	Qty	Type	Description
Raspberry Pi Pico 2 W H	1	Main MCU	MicroPython — steering, motor, sensors, navigation
ESP32-CAM	1	Camera module	RGB565 color detection; UART obstacle reporting to Pico
TB6612FNG	1	Motor driver	PWM + direction control for brushed DC motor
6 V Micro DC Motor	1	Drive motor	Rear wheel propulsion via spur gear drivetrain
HD-1440A Servo	1	Steering servo	PWM-controlled Ackermann front steering
US-100 Ultrasonic	1	Distance sensor	Front wall / corner detection; safety stop
MPU6050	1	IMU / Gyro	Yaw estimation for PID heading and 90° turns
TCS34725	1	Color sensor	Backup / future color validation module
LM2596 / RT3505 Buck	2	Voltage regulator	Buck 1: 5 V logic rail Buck 2: 5–6 V servo rail
2S 450 mAh 40C LiPo	1	Battery	Main power — 7.4 V nominal, 8.4 V full charge
On/Off Switch	1	Power switch	Manual power cut for safe bench and field use
3×10×4 Bearing	4+	Hardware	Front spindles and rear shaft support
M3 Fasteners / Washers	TBD	Hardware	Chassis, mount, and bracket assembly

3.2 Cable & Connection Specifications

3.2.1 Main Power Source & Regulator Inputs

The 2S LiPo battery (7.4 V nominal, 8.4 V fully charged) feeds three destinations through the main on/off switch. The TB6612FNG VM pin receives direct battery voltage — the DC motor therefore runs at 7.4–8.4 V for maximum efficiency.

From	To	Voltage	Peak Current	Notes
2S LiPo (+)	On/off switch (IN)	7.4–8.4 V	2.5 A	Full-load peak across all rails
Switch (OUT)	Buck 1 (IN+)	7.4–8.4 V	0.6 A	Logic rail input
Switch (OUT)	Buck 2 (IN+)	7.4–8.4 V	1.0 A	Servo rail input
Switch (OUT)	TB6612FNG VM	7.4–8.4 V	1.0 A	Direct battery → motor power pin
2S LiPo (–)	Common GND bus	0 V	—	Shared ground for all modules

3.2.2 Buck Converter 1 — Logic Rail (5 V)

From	To	Voltage	Current	Notes
Buck 1 (OUT+)	Pico 2 W H VBUS	5.0 V	50–100 mA	Powers Pico onboard 3.3 V regulator
Buck 1 (OUT+)	ESP32-CAM 5V	5.0 V	100–400 mA	~100 mA idle; up to 400 mA Wi-Fi + flash
Buck 1 (OUT+)	TB6612FNG VCC	5.0 V	1–5 mA	Logic supply only; motor from battery directly

Total Buck 1 output load: approximately 400–500 mA.

3.2.3 Buck Converter 2 — Servo Rail (5 V or 6 V)

From	To	Voltage	Current	Notes
Buck 2 (OUT+)	HD-1440A servo (red)	5.0 V (opt. 6.0 V)	10–1000 mA	~10 mA idle; up to 1 A peak under load or sudden movement

Set Buck 2 to 5 V default. Switch to 6 V for higher torque/speed if wiring supports it.

3.2.4 Pico 3.3 V Sensor Rail

The Pico converts its 5 V VBUS to a clean 3.3 V rail via its onboard regulator and feeds all sensors.

From	To	Voltage	Current	Notes
Pico 3V3 OUT	TCS34725 VCC	3.3 V	~5 mA	Includes onboard LED
Pico 3V3 OUT	MPU6050 VCC	3.3 V	~4 mA	Gyro + accelerometer combined
Pico 3V3 OUT	US-100 VCC	3.3 V	~20 mA	Peak during measurement pulse
Pico GND	All sensor GNDs	0 V	—	Common reference

Total Pico 3.3 V load: ~30–40 mA — well within the 300 mA regulator limit.

3.2.5 Signal & Data Lines

From	To	Voltage	Notes
Pico GPIO 14 (PWM)	HD-1440A servo signal (yellow)	3.3 V	50 Hz PWM, 1–2 ms pulse
Pico GPIO 16 (PWM)	TB6612FNG PWMA	3.3 V	Motor speed control
Pico GPIO 17	TB6612FNG AIN2	3.3 V	Motor direction
Pico GPIO 18	TB6612FNG AIN1	3.3 V	Motor direction
Pico GPIO 19	TB6612FNG STBY	3.3 V	Must be HIGH to enable motor output
Pico GPIO 4 (SDA)	MPU6050 SDA / TCS34725 SDA	3.3 V	Shared I2C data; 4.7 kΩ pull-up
Pico GPIO 5 (SCL)	MPU6050 SCL / TCS34725 SCL	3.3 V	Shared I2C clock; 4.7 kΩ pull-up
Pico GPIO 10 (TRIG)	US-100 TRIG	3.3 V	10 μs trigger pulse
Pico GPIO 11 (ECHO)	US-100 ECHO	3.3 V	US-100 echo is 3.3 V compatible
ESP32-CAM TX	1kΩ+2kΩ divider → Pico GPIO 1 RX	5V → 3.3V	Level-shift required; ~1.6 mA line current
Pico GPIO 0 TX	ESP32-CAM RX	3.3 V	3.3 V within ESP32-CAM range; no shifting needed

3.3 Power Budget

Rail	Consumers	Typical	Peak
Battery direct → TB6612FNG VM	DC Motor	300 mA	1.0 A
Buck 1 — 5 V logic	Pico (100 mA) + ESP32-CAM (400 mA) + TB6612FNG VCC (5 mA)	250 mA	505 mA
Buck 2 — servo rail	HD-1440A Servo	10 mA	1.0 A
Pico 3.3 V out	MPU6050 (4 mA) + TCS34725 (5 mA) + US-100 (20 mA)	29 mA	40 mA
BATTERY TOTAL	All rails combined	~590 mA	~2.5 A

Measure Buck 1 output → set to 5.0 V. Measure Buck 2 output → set to 5.0 V (or 6.0 V for servo). Do not connect any board until both outputs are confirmed with a multimeter.

4 Software Architecture

The software is split between two microcontrollers. The ESP32-CAM runs Arduino C++ firmware that captures RGB565 frames and reports color detections over UART at 115200 baud. The Raspberry Pi Pico 2 W H runs MicroPython scripts that handle motor control, steering PID, sensor fusion, navigation state machines, and lap counting.

File	Platform	Purpose
camera.cpp	ESP32-CAM / Arduino C++	RGB565 color detection for red and green obstacles; UART output
camera_uart_blink.py	Pico / MicroPython	UART receiver test — blinks LED on camera messages
openround.py	Pico / MicroPython	Open-round drive logic with gyro-assisted 90° corner turns
obstacle_round.py	Pico / MicroPython	Obstacle-round: camera tracking, gyro recovery, lap counting
servo_tune.py	Pico / MicroPython	Servo center, direction, and PID steering tuning utility

4.1 Vision System (ESP32-CAM)

The ESP32-CAM captures QQVGA (160×120) RGB565 frames and scans the lower region (ROI_Y_START = 30) with a 2×2 pixel step. Each pixel is converted to RGB888 and tested against threshold rules for red and green. When both colors appear, red is prioritised if its detected area is at least equal to green.

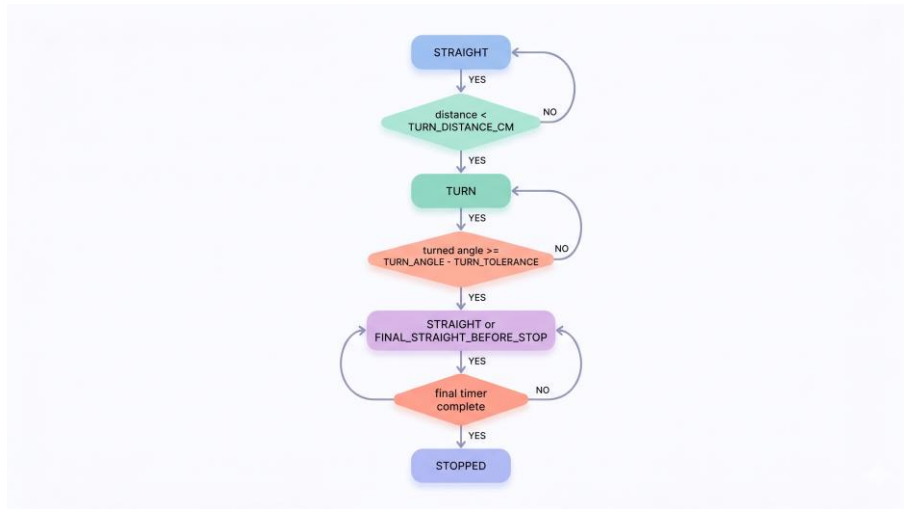
UART output format (one line per frame):

```
RED,<center_x>,<pixel_count>
GREEN,<center_x>,<pixel_count>
NONE
```

Color	Detection Rule
Red	High red channel; low green and blue; red dominates both other channels
Green	High green channel; limited red and blue; green dominates both other channels
Priority	Red prioritised when red area $\geq$ green area
Min area	Must exceed RED_MIN_AREA or GREEN_MIN_AREA to register

4.2 Open Round Navigation

openround.py implements a deterministic state machine for repeated 90° corner turns. A PID loop corrects heading using MPU6050 yaw; the US-100 triggers turn entry when the front wall is closer than TURN_DISTANCE_CM. After STOP_AFTER_TURNS = 16 corners the robot stops.



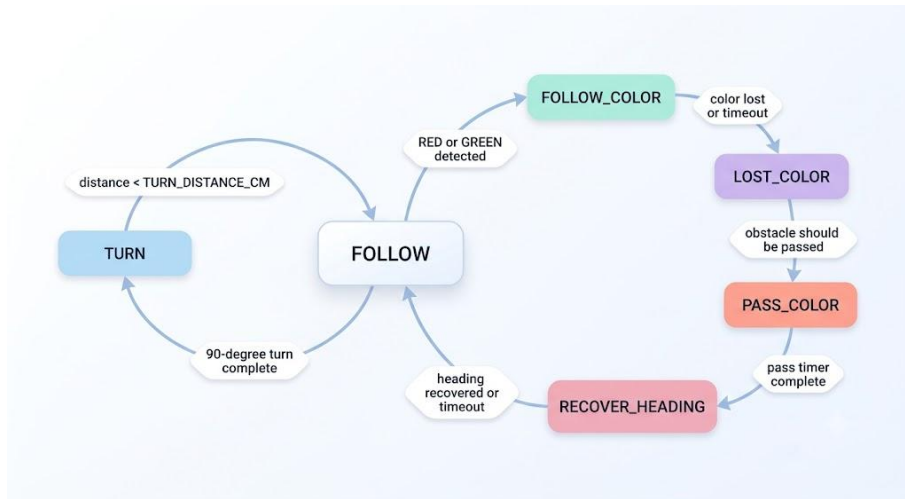
Open Round State Machine

State	Condition to leave	Next state
STRAIGHT	distance < TURN_DISTANCE_CM	TURN
TURN	turned angle ≥ TURN_ANGLE – TURN_TOLERANCE	STRAIGHT or FINAL_STRAIGHT
FINAL_STRAIGHT	final timer complete	STOPPED
STOPPED	—	—

4.3 Obstacle Round Navigation

obstacleround.py extends the base driver for the WRO obstacle round. It combines camera UART detections with gyro heading control, ultrasonic corner detection, and a multi-state obstacle passing routine. The robot tracks corner_count and lap_count; it stops after REQUIRED_LAPS = 4.

Obstacle	Target camera x	Pass side
Red	RED_TARGET_X = 48	Pass on the right
Green	GREEN_TARGET_X = 112	Pass on the left
None visible	—	Gyro-assisted forward drive



Obstacle Round State Machine

Risk	Mitigation
False ultrasonic readings at close range	Distances below MIN_VALID_DISTANCE_CM rejected
Same wall counted twice after turn	TURN_IGNORE_AFTER_TURN_MS blocks immediate re-trigger
Corner trigger during obstacle avoidance	TURN_IGNORE_AFTER_COLOR_MS delays corner detection
Camera briefly loses obstacle	LOST_COLOR keeps short bias from last known color
Two obstacles close together	chain_active extends pass duration and increases bias
Gyro drift after manoeuvres	Yaw and PID state reset after turns and recovery events
UART buffer noise / partial messages	Buffer bounded; lines parsed only after newline termination

4.4 Control System (PID)

Steering uses a discrete PID loop:

```

error      = target_yaw - yaw
integral  += error * dt
derivative = (error - last_error) / dt
output    = (KP * error) + (KI * integral) + (KD * derivative)

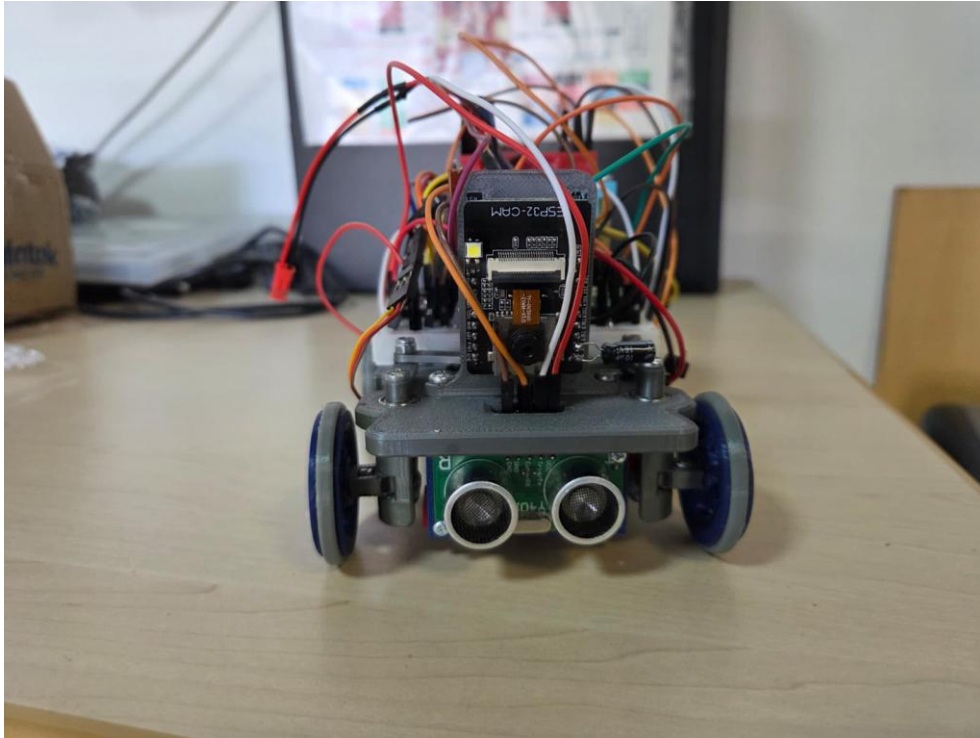
```

Parameter	Open Round	Obstacle Round
SERVO_MIN	30	50
SERVO_CENTER	65	65
SERVO_MAX	100	80
KP	1.8	2.4 (KP_HEADING)
KI	0.00	—
KD	0.25	0.35 (KD_HEADING)
MOTOR_SPEED_STRAIGHT	30	65 (FOLLOW)
MOTOR_SPEED_TURN	50	57
TURN_DISTANCE_CM	—	45
STOP_DISTANCE_CM	—	6

5 Vehicle Photos

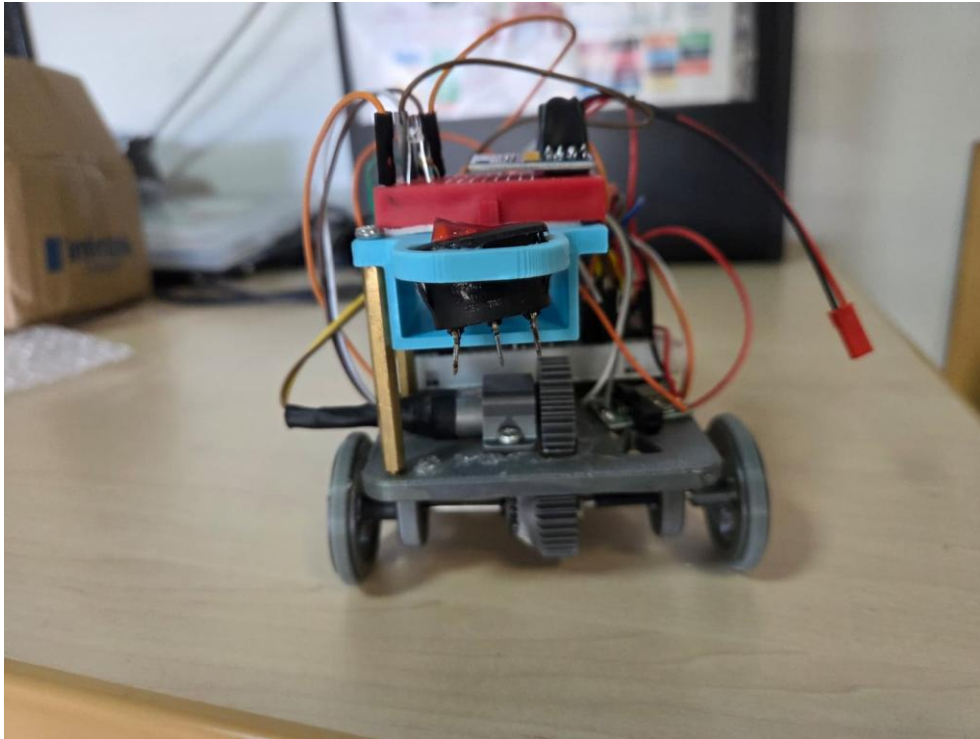
The following photos show the current prototype build. Wiring runs on a breadboard platform and cable routing is not yet finalised. These images reflect the real build state used during development and testing.

Front View



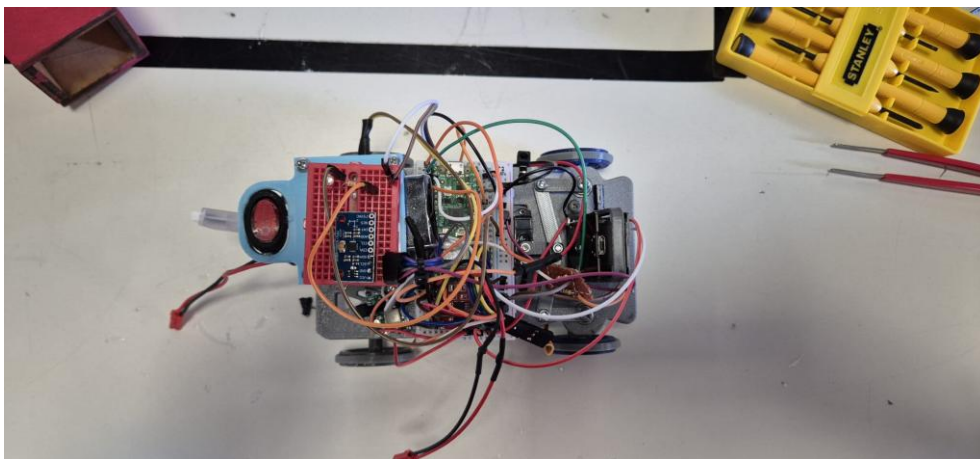
ESP32-CAM on custom bracket at centre; US-100 ultrasonic below chassis nose; blue front wheels with pressed 3×10×4 bearings and O-ring tires.

Rear View



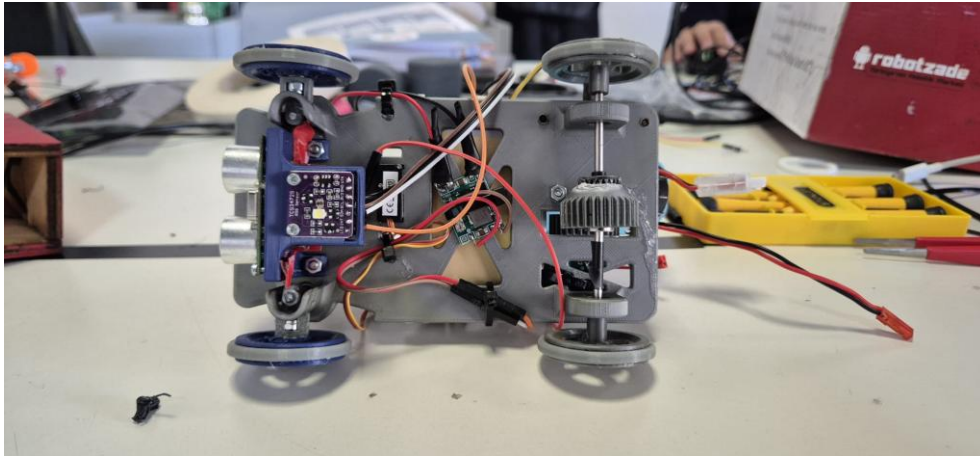
0.8M 31T spur gear visible on rear axle; brushed DC motor in U-bracket; toggle power switch at rear; red JST battery connector.

Top View



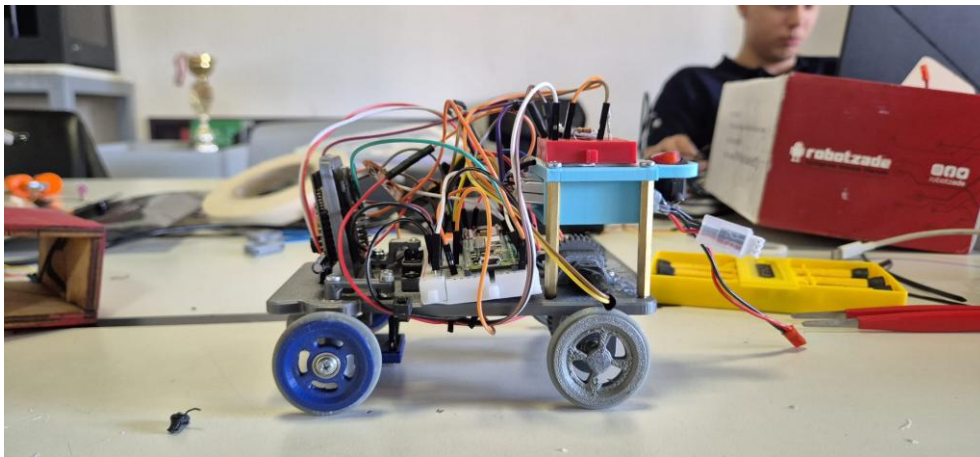
Red breadboard with all prototype circuitry; ESP32-CAM front-left; bamboo support columns elevating electronics layer above chassis.

Bottom View



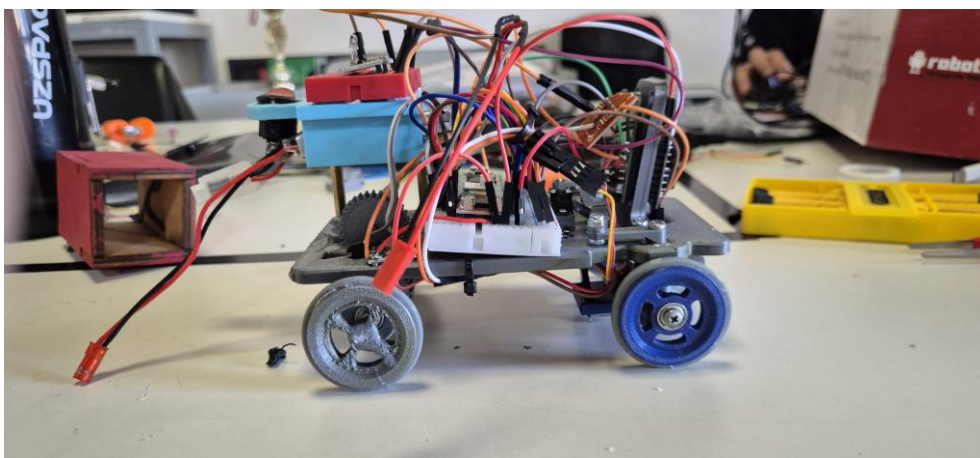
Ackermann steering linkage visible front-left; servo inboard of front axle; 0.8M spur gear pair rear-right; X-pattern lightening pockets throughout.

Left View



Height hierarchy: flat chassis → electronics platform → ESP32-CAM tower. Blue steered front wheel and grey driven rear wheel.

Right View



Clearest view of rear gear train: motor, pinion, and axle driven gear. Front bearing hub vs rear direct shaft hub contrast visible.

6 Validation & Testing

6.1 Mechanical Validation

Test	Procedure	Pass Criteria
Steering range	Servo min→max while wheels lifted	Full travel without binding or wheel/chassis contact
Ackermann behaviour	Observe inner/outer wheel angles	Inner wheel turns more sharply than outer in both directions
Gear mesh	Hand-spin, then low PWM	No skipping, tight spots, or visible gear wobble
Differential action	Hold one wheel, rotate opposite	Allows relative motion without excess friction
Bearing fit	Press bearing, spin wheel	Bearing seated; wheel rotates freely with no lateral play
Sensor mount rigidity	Light shake of chassis	Camera/US-100 angle does not shift
Track clearance	Roll through practice corner	No chassis, wire, or mount scraping

6.2 Electronics Validation

Check	Method	Failure Response
Voltage rails	Multimeter on buck outputs before connecting boards	Do not connect until output corrected
I2C sensors	Confirm MPU6050/TCS34725 addresses detected	Inspect SDA/SCL, power, ground
UART camera	Verify RED/GREEN/NONE messages	Use camera_uart_blink.py for isolated test
Ultrasonic	Compare readings against known distances	Reject below minimum; inspect TRIG/ECHO wiring
Motor polarity	Test with wheels lifted	Swap leads or invert direction logic
Servo direction	Run servo_tune.py	Adjust reverse/settings before autonomous code

6.3 Software / Track Validation

Test	Procedure	Pass Criteria
Camera detection	Show red, green, and no object to ESP32-CAM	Serial output changes between RED, GREEN, NONE within one frame
UART receiver	Run camera_uart_blink.py with camera connected	Red → 1 blink, green → 2 blinks; no spam on sustained colour
Open round	Run openround.py on practice track	Completes 16 corners without double-counting walls
Obstacle round	Run obstacleround.py with red/green obstacles	Red passed on correct side; green passed on correct side; heading recovers
Lap counting	Full 4-lap run	Robot stops after exactly 4 laps

7 Engineering Decisions & Trade-offs

Decision	Why chosen	Trade-off
Separate ESP32-CAM vision module	Offloads camera processing from main Pico; simple UART interface	Requires second board and stable serial wiring; level-shifting for 5→3.3 V
TB6612FNG VM fed directly from battery (7.4–8.4 V)	Maximises motor efficiency; no buck converter losses in motor power path	Motor must tolerate 2S voltage; speed varies slightly across discharge curve
Dual separate buck converters	Isolates motor/servo transients from logic rail; servo gets dedicated current headroom	Two regulators to set and verify; more wiring
1:1 spur gear drivetrain	Preserves full motor RPM for straight-line speed	Lower torque multiplication; relies on low vehicle mass
Differential rear axle	Prevents inside-wheel scrub in corners; improves repeatability	More printed parts; alignment checks required
RGB thresholding instead of ML	Runs at full frame rate on ESP32-CAM; easy to tune on track	Sensitive to lighting; threshold calibration needed per venue
Gyro-based 90° turn completion	More repeatable than timing-only turns	Requires startup calibration; drift handling after major manoeuvres
US-100 for corner detection	Simple, direct front-wall measurement; cheap and reliable	Needs filtering for close-range noise readings
State machine control	Easy to debug and explain during judging	More states require careful transition timing
Flat-plate chassis with X-pattern cutouts	Low CG; easy to add mounting points; weight reduction without sacrificing stiffness	Less inherent rigidity than a box frame; layer orientation matters for print strength
Ackermann steering linkage	Eliminates tire scrub in corners; improves repeatability	4 parts to assemble and align; geometry depends on print accuracy
O-ring tires	Simple replacement; consistent geometry batch-to-batch; no gluing	Limited traction tuning vs custom moulded compounds
run_log.txt logging	Allows post-run diagnosis without live serial monitoring	MicroPython filesystem writes can be slow; log rate must not block control loop

This document covers the complete engineering evidence for Team CYBERRCORE's WRO 2026 Future Engineers robot — mechanical design, electronics architecture, wiring specifications, software algorithms, and validation procedures.